



Artificial Intelligence Software, Lab 1

Reproducible environment setup:
Conda, UV, Docker



Center for
Cognitive
Modeling



Common Labs Workflow

Lab X

1. Oleg explains some new concepts.
2. Oleg provides HW, which should be done after the lab.
3. You start to solve new HW. You can do it at home.

Between lab X and lab X+1

1. You should finish lab tasks and solve HW.
2. Submit HW.

Lab X+1

1. Oleg explains some new concepts.
2. Oleg provides new HW, which should be done after the lab.
3. You defend previous HW solutions and results.
4. You start to solve new HW. You can do it at home.

The "Works on My Machine" Syndrome

Why do we care about environment?

The Problem: Code works locally but fails in production or on a colleague's PC.

Root Causes:

- **OS Mismatch:** Windows vs Ubuntu
- **System Libraries:** Different versions of glibc, ffmpeg, cuda.
- **Shadow Dependencies:** "I just installed it globally a year ago".

Solution: Explicitly defined, isolated, and reproducible environments.





The Reproducibility Spectrum

1. **Level 0: Global Scope** (Chaos). System python, global pip.
2. **Level 1: Isolation** (venv). Separate folders for packages, but versions may float.
3. **Level 2: Determinism** (poetry.lock, uv.lock). Exact versions and hashes of all transitive dependencies.
4. **Level 3: System Deps** (conda, pixi). Python + CUDA + Binary tools.
5. **Level 4: OS Snapshot** (docker). Complete user-space filesystem isolation.



The "Lock File" Concept

Technically enforcing reproducibility

The Challenge: You ask for pandas. It needs numpy, pytz, etc. Tomorrow, a new version of numpy is released and breaks your code.

The Solution: Locking.

- **Manifest** (pyproject.toml): Abstract requirements ("I need pandas roughly version 2.0").
- **Lock File:** Concrete Snapshot. Contains exact versions and cryptographic hashes of *all* transitive dependencies.

Rule: Always commit the lock file to Git!

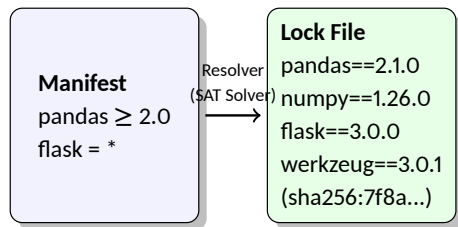


Figure: Resolution Process



Tool 1: uv (The Speedster)

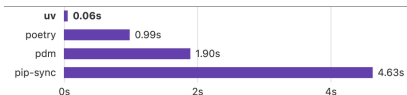
Modern replacement for pip, venv, and pip-tools

Core Features:

- Written in **Rust**. Extremely fast (10-100x faster than pip).
- Unified tool: Manages python versions, virtual envs, and packages.
- **Deterministic**: Generates uv.lock for reproducible builds.

Best For:

- CI/CD Pipelines (caches are heavily optimized).
- Rapid prototyping and scripting.



Benchmark: Package Installation Time



Why Speed Matters?

CI/CD Costs & Developer Experience

1. Developer Flow:

- Waiting for dependencies breaks mental context.
- "Context switching" is expensive.

2. Continuous Integration (CI):

- Clean install happens on every Pull Request.
- **Scenario:** 10 devs, 5 PRs/day.
 - Tool A (5 min): 250 mins/day wasted.
 - Tool B (10 sec): 8 mins/day wasted.



Tool 2: Poetry (The Developer Standard)

Dependencing Management & Packaging

Core Features:

- Uses pyproject.toml (PEP 518 standard).
- **Strict Resolver:** Guarantees no conflicting dependencies.
- Handles packaging and publishing to PyPI.

Best For:

- Developing Python Libraries.
- Projects where strict dependency logic is crucial.

pyproject.toml:

```
[tool.poetry]
name = "my-lib"
version = "0.1.0"

[tool.poetry.dependencies]
python = "^3.10"
numpy = "^1.26"
pandas = ">=2.0"

[tool.poetry.group.dev]
pytest = "^7.4"
```




Tool 3: Conda (The Scientific Heavyweight)

Binary Package Management

Core Features:

- ****Language Agnostic****: Installs Python, C++, CUDA, R, etc.
- ****Pre-compiled Binaries****: avoiding "GCC compilation failed" errors.
- Solves deep system dependencies (e.g., specific glibc versions).

Best For:

- Data Science & ML (PyTorch, TensorFlow).
- Working with non-Python libraries (GDAL, FFMPEG).



Tool 4: Pixi (The Modern Hybrid)

Conda power with Rust speed

Core Features:

- Uses **conda-forge** ecosystem (access to all binary packages).
- **Project-Local**: Installs everything in .pixi folder (no global env pollution).
- Works like npm or cargo. Cross-platform lock-files.

Best For:

- Reproducible ML Projects (Sharing code that "just works").
- Robotics (installing ROS packages via boa/conda).



Docker: Shipping the Computer

Solving the "Matrix from Hell"

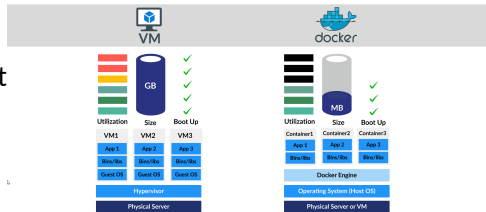
Concept: Package your application with everything it needs to run (Code + Runtime + System Tools + System Libs).

Why?

- Solves "Matrix of Hell" (OS × App version).
- Immutable Infrastructure: The image you test is the image you deploy.

Robotics/ML Specifics:

- **GPU Access:** nvidia-container-toolkit
- **GUI:** X11 forwarding to see rviz or plots.
- **Hardware:** --network host, --privileged.





Docker: Multi-stage Builds

Idea: Use intermediate images to build items, copy only artifacts to the final image.

Example:

- *Stage 1 (Builder):* Large image with gcc, cmake, headers. Compiles code. (Size: 2GB)
- *Stage 2 (Runtime):* Minimal OS + Binary. (Size: 100MB)

Pros:

- Minimal prod image size.
- Security (no compilers/keys in prod).

Cons:

- Dockerfile complexity.
- Harder to debug build issues in prod image.



Orchestration: Docker Compose

Managing Multi-Container Robotics Systems

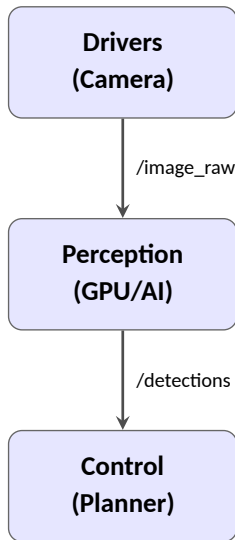
Why?: Managing multiple containers with CLI flags is painful.

Role: "Infrastructure as Code" for local development.
Define services, networks, and volumes in one YAML file.

Microservices in Robotics (ROS 2):

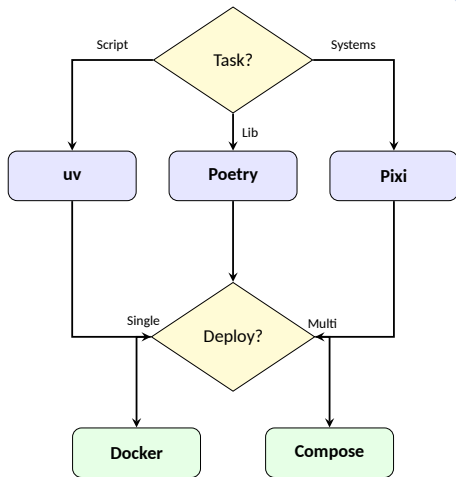
- *Drivers Container:* Hardware access.
- *Perception Container:* PyTorch/CUDA environment.
- *Planning Container:* C++ environment.

Benefit: Isolation. Crash in Vision node doesn't kill the Motor controller.



Summary: Strategy Selection

- Just a script / Speed matter? → **uv**
- Developing Python Library? → **Poetry**
- Data Science with complex binaries? → **Pixi**
(or Conda)
- Need to share with others / Deploy? → **Docker**
- Multiple services? → **Docker Compose**



Decision Flowchart

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521